

МАШИННОЕ ОБУЧЕНИЕ В ДЕТАЛЯХ

Техническая стратегия для инженеров в области ИИ в эпоху глубокого обучения

Эндрю Нг

Проект DeepLearning.AI

Полный перевод первой половины книги (Главы 1–29)

© 2018 Andrew Ng. Все права защищены.

Оглавление (Полная книга, Главы 1–58)

1 Почему важна стратегия машинного обучения	4
2 Как использовать эту книгу, чтобы помочь вашей команде	5
3 Предварительные требования и обозначения	6
4 Масштаб как двигатель прогресса в машинном обучении	7
Настройка выборок для разработки (dev) и тестирования (test)	9
5 Ваши выборки для разработки и тестирования	10
6 Выборки dev и test должны приходить из одного распределения	12
7 Насколько большими должны быть выборки dev/test?	14
8 Установите единую числовую метрику оценки для оптимизации	15
9 Оптимизирующие и удовлетворяющие метрики	17
10 Наличие dev-выборки и метрики ускоряет итерации	19
11 Когда следует менять dev/test выборки и метрики	20
12 Ключевые выводы: Настройка dev- и test-выборок	22
Базовый анализ ошибок	23
13 Быстро постройте первую систему, затем переходите к итерациям	24
14 Анализ ошибок: Изучение примеров из dev-выборки для оценки идей	25
15 Параллельная оценка нескольких идей при анализе ошибок	27
16 Очистка неверно размеченных примеров в dev- и test-выборках	29
17 При большой dev-выборке разделите ее на две части: визуальную и скрытую	31
18 Какого размера должны быть визуальная (Eyeball) и скрытая (Blackbox) выборки?	33
19 Ключевые выводы: Базовый анализ ошибок	34
Смещение и дисперсия	35
20 Смещение и дисперсия: два главных источника ошибок	36
21 Примеры смещения и дисперсии	38
22 Сравнение с оптимальным уровнем ошибки	39
23 Борьба со смещением и дисперсией	41
24 Компромисс между смещением и дисперсией	42
25 Методы снижения устранимого смещения	43
26 Анализ ошибок на обучающей выборке	44
27 Методы снижения дисперсии	45
Кривые обучения	46
28 Диагностика смещения и дисперсии: Кривые обучения	47
29 Построение кривой ошибки на обучении	49
Сравнение с уровнем человеческого качества	37

30 Интерпретация кривых обучения: Высокое смещение	38
31 Другие случаи интерпретации кривых	38
32 Практические аспекты построения кривых обучения	39
33 Почему мы сравниваем успехи моделей с человеком	40
34 Как определить уровень человеческого качества	40
35 Превосходство над человеком	41
Обучение и тестирование на различных распределениях	42
36 Сценарии несовпадения распределений	43
37 Стоит ли использовать все доступные данные	43
38 Проблема противоречивых данных	44
39 Взвешивание данных при обучении	44
40 Обобщение с обучения на dev-выборку	45
41 Идентификация типов ошибок и несоответствия данных	45
42 Способы преодоления data mismatch	46
43 Искусственный синтез данных	46
Отладка алгоритмов логического вывода	47
44 Тест верификации оптимизации	48
45 Общая форма теста верификации	48
46 Применение в обучении с подкреплением	49
Сквозное глубокое обучение	50
47 Становление сквозного обучения	51
48 Дополнительные примеры сквозных систем	51
49 Плюсы и минусы сквозного подхода	52
50 Выбор компонентов пайплайна: Доступность данных	53
51 Выбор компонентов пайплайна: Простота подзадач	53
52 Прямое обучение сложным комплексным выходам	54
Покомпонентный анализ ошибок пайплайна	55
53 Анализ ошибок по частям системы	56
54 Точное приписывание вины конкретному модулю	56
55 Общий случай покомпонентного анализа ошибок	57
56 Сравнение компонентов с человеческим уровнем	57
57 Как обнаружить фундаментальные дефекты архитектуры пайплайна	58
58 Построение команды супергероев	59

Why Machine Learning Strategy / Почему важна стратегия машинного обучения

Машинное обучение лежит в основе огромного количества важнейших современных приложений, таких как веб-поиск, защита от спама, распознавание речи, рекомендательные системы и многие другие. Я предполагаю, что вы или ваша команда сейчас работаете над таким приложением и стремитесь достичь быстрого прогресса. Эта книга призвана помочь вам в этом.

Пример: Создание стартапа с фотографиями кошек

Представьте, что вы запускаете стартап, который будет радовать любителей кошек бесконечным потоком их фотографий. Для обнаружения кошек на изображениях вы решаете использовать нейросеть, построив систему компьютерного зрения. Однако точность вашего алгоритма пока оставляет желать лучшего. На вас оказывается колоссальное давление с целью как можно быстрее доработать детектор кошек. Что вы предпримете?

У вашей команды масса идей, например:

- Собрать больше данных: найти дополнительные фотографии кошек.
- Собрать более разнообразную обучающую выборку. Например, фотографии кошек в необычных позах, кошек редкого окраса, а также снимки, сделанные с различными настройками камеры.
- Обучать алгоритм дольше, выполнив большее количество итераций градиентного спуска.
- Использовать более крупную нейросеть (с большим числом слоев, скрытых нейронов или параметров).
- Попробовать уменьшить размер нейросети.
- Добавить регуляризацию (например, L2-регуляризацию).
- Изменить архитектуру нейросети (функцию активации, количество скрытых блоков и т.д.).

Если вы выберете правильное направление среди этих вариантов, вы создадите лучшую платформу с картинками кошек и приведете компанию к успеху. Ошибившись, вы рискуете впустую потратить месяцы работы. Как поступить?

Эта книга ответит на данный вопрос. Большинство проблем в машинном обучении оставляют определенные зацепки, которые указывают на то, какие шаги принесут пользу, а какие окажутся

бесполезными. Научившись распознавать эти подсказки, вы сэкономите месяцы или даже годы драгоценного времени разработки.

ГЛАВА 2

How to use this book to help your team / Как использовать эту книгу для помощи команде

Прочитав эту книгу, вы получите глубокое понимание того, как определять техническое направление развития проектов в области машинного обучения.

Однако ваши коллеги могут не сразу понять, почему вы рекомендуете именно этот путь. Например, вы хотите внедрить единую числовую метрику оценки работы алгоритма, а команда не до конца уверена в целесообразности этого шага. Как их убедить?

Именно поэтому я сделал главы этой книги очень короткими. Вы можете распечатать их и дать коллегам прочитать буквально 1–2 страницы, содержащие ровно ту информацию, которая необходима в данный момент.

Небольшие коррективы в приоритизации задач способны кардинально повысить продуктивность всей команды. Помогая своим коллегам внедрять такие изменения, вы сможете стать настоящим супергероем для своей команды!

Prerequisites and Notation / Предварительные требования и обозначения

Если вы уже проходили базовый курс по машинному обучению (например, мой массовый открытый онлайн-курс по машинному обучению на платформе Coursera) или имеете практический опыт применения обучения с учителем (supervised learning), вы без труда поймете этот текст.

Я предполагаю, что вы знакомы с концепцией обучения с учителем: построением функции, отображающей входные данные x в целевые значения y на основе размеченных обучающих примеров (x, y) . К алгоритмам обучения с учителем относятся линейная регрессия, логистическая регрессия и нейронные сети. Существует множество направлений машинного обучения, но сегодня основная часть его практической ценности создается именно благодаря обучению с учителем.

Я буду часто ссылаться на нейронные сети (также известные как «глубокое обучение»). Для понимания книги вам потребуются лишь базовые представления о принципах их работы.

Если вы не знакомы с упомянутыми понятиями, рекомендую посмотреть видеоматериалы первых трех недель курса машинного обучения на Coursera по адресу: <http://ml-class.org>.

Scale drives machine learning progress / Масштаб как двигатель прогресса

Многие идеи, лежащие в основе глубокого обучения (нейросетей), были сформулированы десятилетия назад. Почему же они совершают прорыв именно сейчас?

Двумя мощнейшими катализаторами этого прогресса стали:

- **Доступность данных.** Люди проводят всё больше времени с цифровыми устройствами (компьютерами, смартфонами). Их цифровая активность генерирует огромные объемы данных, которые мы можем использовать для обучения наших алгоритмов.
- **Масштаб вычислений.** Всего несколько лет назад мы получили техническую возможность обучать нейросети достаточно большого размера, способные эффективно осваивать те гигантские наборы данных, которыми мы располагаем.

В деталях: при накоплении данных качество работы традиционных алгоритмов обучения (таких как логистическая регрессия) обычно выходит на плато. Это означает, что кривая обучения выравнивается и алгоритм перестает улучшаться, сколько бы дополнительных данных мы ему ни давали.

Традиционные алгоритмы вели себя так, словно они просто не знали, что делать со всем тем объемом данных, который появился в нашем распоряжении.

Если же вы обучите небольшую нейронную сеть (Small NN) на той же задаче обучения с учителем, вы получите чуть более высокое качество работы:

По мере дальнейшего увеличения размера нейронных сетей — перехода к средним (Medium NN) и большим нейросетям (Large NN) — качество работы продолжает расти:

Таким образом, наилучших результатов сегодня удастся достичь при сочетании двух факторов: (i) обучения очень крупной нейросети и (ii) наличия огромного массива данных. Архитектура сети также важна, но увеличение масштаба сети и объема собираемых данных остается самым надежным путем повышения качества работы алгоритма.

SETTING UP DEVELOPMENT AND TEST SETS

НАСТРОЙКА ВЫБОРОК ДЛЯ РАЗРАБОТКИ И

ТЕСТИРОВАНИЯ

ГЛАВА 5

Your development and test sets / Ваши выборки для разработки и тестирования

Вернемся к нашему примеру с приложением для любителей кошек. Пользователи вашего мобильного приложения загружают фотографии самых разных объектов. Ваша задача — автоматически находить среди них изображения кошек.

Ваша команда формирует большой обучающий набор данных, скачивая фотографии кошек (положительные примеры) и других объектов (отрицательные примеры) с различных веб-сайтов. Они делят эти данные в классической пропорции 70% на 30% на обучающую (train) и тестовую (test) выборки. На этих данных они строят детектор кошек, который отлично показывает себя как на обучении, так и на тесте.

Но после запуска классификатора внутри мобильного приложения вы с ужасом обнаруживаете, что качество его работы крайне низкое!

Что пошло не так?

Вы выясняете, что фотографии, загружаемые пользователями, визуально сильно отличаются от картинок с сайтов, из которых состояла обучающая выборка. Пользователи делают снимки на камеры мобильных телефонов, поэтому они часто имеют низкое разрешение, смазаны и плохо освещены. Поскольку ваши обучающая и тестовая выборки состояли исключительно из качественных картинок с веб-сайтов, алгоритм не смог обобщить закономерности на реальное распределение данных, которое вас интересует — мобильные фотографии.

До эпохи больших данных общепринятым стандартом в машинном обучении было случайное деление имеющихся данных в пропорции 70% на 30%. Этот подход жизнеспособен, но он становится всё более ошибочным в современных приложениях, где распределение обучающих данных (картинки с сайтов) отличается от распределения данных, на которых система должна работать в реальности (мобильные фото).

Обычно мы определяем выборки следующим образом:

- **Обучающая выборка (Training set)** — данные, на которых запускается алгоритм обучения.
- **Выборка для разработки (Dev / development set)** — данные, используемые для настройки параметров, отбора признаков и принятия других архитектурных решений по алгоритму. Ее также называют валидационной выборкой (hold-out cross validation set).

- **Тестовая выборка (Test set)** — данные, применяемые исключительно для итоговой оценки качества работы алгоритма, но не для принятия решений о выборе самого алгоритма или его параметров.

Цель dev- и test-выборок — направлять вашу команду к наиболее важным изменениям в системе машинного обучения.

Правило: Выбирайте dev- и test-выборки так, чтобы они максимально точно отражали те данные, которые вы ожидаете получать в будущем и на которых вы хотите работать качественно.

ГЛАВА 6

Your dev and test sets should come from the same distribution / Выборки dev и test должны соответствовать одному распределению

Представьте, что вы разделили изображения кошек в вашем приложении на четыре сегмента по географическому признаку крупнейших рынков: (i) США, (ii) Китай, (iii) Индия и (iv) Другие страны. Чтобы сформировать dev- и test-выборки, вы решаете отправить данные из США и Индии в выборку dev, а данные из Китая и других стран — в test. Разумный ли это подход?

Нет! Как только вы определили dev-выборку, ваша команда начинает целенаправленно оптимизировать качество работы алгоритма именно на ней. Соответственно, dev-выборка должна отражать общую задачу, которую вы хотите решить: хорошо работать во всех четырех регионах, а не только в двух из них.

Вторая проблема разного распределения данных в dev и test заключается в высоком риске того, что команда создаст систему, которая блестяще покажет себя на dev, но полностью провалится на тестовой выборке. Это приводит к колоссальному разочарованию и впустую потраченным ресурсам. Не допускайте этого.

Если бы выборки dev и test принадлежали к одному распределению, то при неудовлетворительном результате на тесте диагноз был бы очевиден: вы переобучились под dev-выборку. Решением в данном случае является сбор дополнительных данных для dev-выборки. Но если распределения разные, ваши варианты менее понятны: (1) Вы переобучились под выборку dev. (2) Тестовая выборка объективно сложнее. (3) Тестовая выборка просто другая, поэтому методы оптимизации для dev оказались бесполезными для test.

How large do the dev/test sets need to be? / Насколько большими должны быть выборки dev/test?

Размер выборки dev должен быть достаточным для того, чтобы улавливать разницу между тестируемыми вами алгоритмами. Например, если классификатор А имеет точность 90.0%, а классификатор Б — 90.1%, то выборка dev из 100 примеров физически не позволит вам обнаружить это улучшение в 0.1%. В большинстве практических задач выборки dev размером от 1 000 до 10 000 примеров являются стандартом. С набором в 10 000 примеров вы с высокой вероятностью заметите прогресс даже в 0.1%.

Что касается тестовой выборки, ее объем должен быть достаточным для обеспечения высокой уверенности в итоговой оценке работы всей системы. Историческое правило выделять под тест 30% данных отлично работает при скромных объемах выборки (от 100 до 10 000 примеров). Однако в эпоху больших данных, когда счет примеров идет на сотни миллионов и миллиарды, доля данных, выделяемых под dev/test, стремительно сокращается, хотя их абсолютное количество растет. Нет никакого смысла раздувать выборки dev и test сверх того объема, который реально необходим для надежной оценки работы алгоритмов.

ГЛАВА 8

Establish a single-number evaluation metric for your team to optimize /

Установите единую числовую метрику оценки

Точность классификации (accuracy) — классический пример единой числовой метрики. Вы прогоняете классификатор на dev-выборке и получаете одно конкретное число. На основе этой метрики легко сделать вывод: если модель А показала точность 97%, а модель Б — 90%, то модель А лучше.

В отличие от нее, Точность (Precision) и Полнота (Recall) не являются единой метрикой, так как дают сразу два числа для оценки. Это сильно усложняет сравнение алгоритмов. Предположим, ваши модели показывают следующие результаты:

Классификатор	Точность (Precision)	Полнота (Recall)
А	95%	90%
В	98%	85%

В данном случае ни один из классификаторов не имеет очевидного преимущества, что затрудняет выбор оптимального решения.

Если для вас важны оба параметра, я настоятельно рекомендую объединить их в одно число с помощью стандартных математических методов, таких как F1-мера (среднее гармоническое):

Классификатор	Точность (Precision)	Полнота (Recall)	F1-мера (F1 score)
А	95%	90%	92.4%
В	98%	85%	91.0%

Наличие единой числовой метрики колоссально ускоряет процесс принятия решений при выборе лучшей модели из десятков альтернативных вариантов. Она дает четкий ранжированный список предпочтений и ясное направление движения вперед.

ГЛАВА 9

Optimizing and satisficing metrics / Оптимизирующие и удовлетворяющие метрики

Вот еще один эффективный способ сочетания нескольких критериев оценки. Предположим, вы заботитесь как о точности алгоритма, так и о времени его выполнения (инференсе):

Классификатор	Точность (Accuracy)	Время выполнения (Running time)
A	90%	80 мс
B	92%	95 мс
C	95%	1,500 мс

Кажется противоестественным объединять точность и время выполнения в единую искусственную формулу, например: ***Accuracy - 0.5 times RunningTime***.

Вместо этого вы можете сделать следующее: сначала определите, какое время выполнения является «приемлемым». Допустим, всё, что работает быстрее 100 мс, является приемлемым. Затем максимизируйте точность при условии, что ваш классификатор удовлетворяет этому критерию времени. В данном сценарии время выполнения является **удовлетворяющей метрикой (satisficing metric)**, а точность — **оптимизирующей метрикой (optimizing metric)**.

ГЛАВА 10

Having a dev set and metric speeds up iterations / Наличие dev-выборки и метрики ускоряет итерации

Крайне трудно заранее предугадать, какой подход лучше всего сработает для новой задачи. Построение системы машинного обучения — это высокоитеративный процесс:

Каждый раз, когда у вас появляется идея, реализация ее в коде и измерение ее производительности на dev-выборке позволяют мгновенно понять, движетесь ли вы в правильном направлении. Без чёткой метрики и выборки вам пришлось бы часами тестировать приложение вручную, что критически замедлило бы прогресс.

When to change dev/test sets and metrics / Когда следует менять dev/test выборки и метрики

В начале нового проекта я стараюсь максимально быстро сформировать первоначальные выборки dev/test и метрику — обычно менее чем за одну неделю. Лучше начать с несовершенного варианта, чем увязнуть в размышлениях.

Если позже вы поймете, что промахнулись, обязательно измените их. Есть три основные причины, по которым метрика или dev-выборка могут неверно оценивать модели:

1. **Реальное распределение изменилось:** например, пользователи начали загружать фотографии маленьких котят вместо взрослых кошек. Вам нужно обновить выборки, чтобы они стали репрезентативными.
2. **Вы переобучились под выборку dev:** если результаты на dev значительно превосходят результаты на тестовой выборке, это явный признак переобучения под валидационный набор. Вам нужна свежая выборка dev.
3. **Метрика измеряет не то, что нужно продукту:** например, модель А точнее модели Б, но периодически пропускает нежелательные изображения. Нужно переписать метрику, чтобы она жестко штрафовала такие критические ошибки.

Takeaways: Setting up development and test sets / Ключевые выводы по настройке выборок

- Выбирайте выборки dev и test из распределения, которое отражает ваши будущие реальные данные.
- По возможности следите, чтобы выборки dev и test принадлежали к одному распределению.
- Установите единую числовую метрику оценки для оптимизации работы команды. При наличии нескольких целей выделите одну оптимизирующую и остальные удовлетворяющие метрики.
- Машинное обучение — итеративный процесс. Наличие чётких выборок и метрик позволяет быстрее отбрасывать плохие идеи и развивать успешные.

BASIC ERROR ANALYSIS

БАЗОВЫЙ АНАЛИЗ ОШИБОК

ГЛАВА 13

Build your first system quickly, then iterate / Быстро постройте первую систему, затем переходите к итерациям

Вы хотите создать новую систему защиты от спама. У вашей команды множество идей: сбор гигантских данных через ловушки («honeypots»), извлечение сложных текстовых признаков, анализ сетевых заголовков и серверов. С чего начать?

Не пытайтесь сразу спроектировать и построить идеальную систему. Вместо этого соберите базовую рабочую модель всего за несколько дней. Даже если она будет далека от совершенства, ее работа даст вам неоценимые зацепки и укажет самые перспективные направления для инвестиций вашего времени.

Error analysis: Look at dev set examples to evaluate ideas / Изучение примеров для оценки идей

Вы заметили, что ваш детектор кошек иногда принимает собак за кошек. Коллега предлагает потратить месяц на интеграцию стороннего софта для распознавания собак. Стоит ли это делать?

Прежде чем тратить месяц, проведите анализ ошибок. Возьмите 100 случайных примеров из dev-выборки, на которых алгоритм ошибся, и вручную посчитайте, сколько среди них собак. Если собак всего 5%, то максимальный выигрыш от этого проекта составит всего 5% от ваших ошибок (например, снижение общей ошибки с 10% до 9.5%). Проект не стоит потраченного месяца. Но если собак окажется 50%, этот проект имеет наивысший приоритет.

ГЛАВА 15

Evaluating multiple ideas in parallel during error analysis / Параллельная оценка нескольких идей

Вы можете эффективно оценивать множество различных идей параллельно, создав простую таблицу в Excel во время ручного просмотра ~100 ошибочных изображений из dev-выборки:

Изображение	Собака (Dog)	Крупные кошачьи (Great cat)	Размытые (Blurry)	Комментарии
1	✓			Необычный окрас питбуля
2			✓	Размытое фото на ходу
3		✓		Лев; фото из зоопарка в дождливый день
4		✓	✓	Пантера за деревом; размыто
% от общего числа	8%	43%	61%	

Из этой таблицы сразу видно, что борьба с размытыми изображениями (61%) и крупными дикими кошками (43%) принесет гораздо больше пользы, чем исправление ошибок с собаками (всего 8%).

Cleaning up mislabeled dev and test set examples / Очистка неверно размеченных примеров

Во время анализа ошибок вы можете обнаружить, что некоторые изображения были неверно размечены самими людьми-разметчиками еще до того, как их увидел алгоритм. Добавьте столбец «Mislabeled» (Неверная разметка) в свою таблицу анализа ошибок. Стоит ли тратить время на их исправление?

Если общая ошибка вашей модели составляет 10%, а ошибки разметки составляют всего 0.6% от этого объема, их исправление не критично. Но если вы улучшили модель до общей ошибки в 2.0%, а ошибки разметки по-прежнему составляют 0.6% (то есть целых 30% от всех ваших текущих ошибок!), пришло время вручную переразметить данные в dev- и test-выборках, чтобы ваши оценки оставались точными.

If you have a large dev set, split it into two subsets / Разделение большой dev-выборки на Eyeball и Blackbox

Если у вас огромная dev-выборка (например, 5 000 примеров) и модель ошибается на 1 000 из них, просмотреть их все вручную невозможно. Разделите dev-выборку на две части:

1. **Визуальная выборка (Eyeball dev set):** небольшая часть (например, 10-20% от ошибок, около 100 штук), которую вы будете детально изучать глазами для анализа ошибок.
2. **Скрытая выборка (Blackbox dev set):** оставшаяся большая часть, которую вы никогда не смотрите глазами. Она используется для автоматической оценки моделей и настройки гиперпараметров.

Это предотвратит проблему скрытого переобучения вашего собственного человеческого восприятия под те примеры, которые вы постоянно разглядываете при анализе.

ГЛАВА 18

How big should the Eyeball and Blackbox dev sets be? / Какого размера должны быть Eyeball и Blackbox выборки?

Ваша визуальная выборка (Eyeball dev set) должна быть достаточно большой, чтобы содержать хотя бы 50–100 ошибок для анализа. Если уровень ошибок вашего классификатора составляет 5%, то чтобы получить 100 ошибок, размер Eyeball выборки должен быть около 2 000 примеров (**$0.05 \times 2000 = 100$**). Скрытая выборка (Blackbox dev set) стандартно составляет от 1 000 до 10 000 примеров для обеспечения высокой статистической надежности автоматических тестов.

Takeaways: Basic error analysis / Ключевые выводы по базовому анализу ошибок

- Быстро постройте простую систему на старте проекта, чтобы использовать анализ ошибок для приоритизации дальнейших задач.
- Проводите анализ ошибок вручную на ~100 примерах, подсчитывая проценты по различным категориям проблем.
- При необходимости разделяйте выборку на Eyeball и Blackbox подмножества, чтобы избежать переобучения аналитика под данные.

BIAS AND VARIANCE

СМЕЩЕНИЕ И ДИСПЕРСИЯ

Bias and Variance: The two big sources of error / Смещение и дисперсия — два главных источника ошибок

В машинном обучении существуют два ключевых источника ошибок: **смещение (bias)** и **дисперсия (variance)**. Понимание этих концепций — ключ к эффективному управлению временем разработки.

Представим, что ваша цель — создать детектор кошек с уровнем ошибки 5%. В данный момент ошибка на обучении составляет 15%, а на dev-выборке — 16%. В этой ситуации сбор дополнительных данных не даст практически ничего. Ваша главная проблема — алгоритм плохо обучается даже на тех данных, которые у него уже есть. Сначала необходимо снизить смещение (улучшить показатели на обучении).

Мы разбиваем ошибку на dev-выборке на две составляющие:

- **Смещение (Bias):** Ошибка на обучающей выборке (в данном примере 15%).
- **Дисперсия (Variance):** Разница между ошибкой на dev-выборке и ошибкой на обучении (в данном примере $16\% - 15\% = 1\%$).

Некоторые изменения в алгоритме помогают бороться со смещением (например, увеличение размера сети), другие — с дисперсией (например, добавление регуляризации или данных).

Examples of Bias and Variance / Примеры смещения и дисперсии

Рассмотрим различные сценарии для задачи классификации кошек (целевой уровень ошибки равен ~0%):

- **Ошибка на обучении = 1%, Ошибка на dev = 11%:** Высокая дисперсия (High Variance). Модель переобучилась под обучающую выборку (overfitting) и плохо обобщает на новые данные.
- **Ошибка на обучении = 15%, Ошибка на dev = 16%:** Высокое смещение (High Bias). Модель недообучилась (underfitting), она плохо улавливает закономерности даже в обучающих данных.
- **Ошибка на обучении = 15%, Ошибка на dev = 30%:** Высокое смещение и высокая дисперсия одновременно. Алгоритм плохо работает на обучении и еще в два раза хуже на валидации.
- **Ошибка на обучении = 0.5%, Ошибка на dev = 1%:** Низкое смещение, низкая дисперсия. Идеальный результат!

Comparing to the optimal error rate / Сравнение с оптимальным уровнем ошибки

Понятие «высокого смещения» всегда относительно и зависит от минимально возможного уровня ошибки для конкретной задачи (ошибки Байеса). Предположим, вы работаете над распознаванием речи в условиях сильного шума, где даже человек ошибается в 14% случаев. Оптимальный уровень ошибки равен 14%.

Если ваша модель показывает: Ошибка на обучении = 15%, Ошибка на dev = 30%, то:

- **Неустраняемое смещение (Optimal error rate):** 14%.
- **Устранимое смещение (Avoidable bias):** $15\% - 14\% = 1\%$. У вас практически нет проблем со смещением! Модель работает почти на пределе возможностей теоретического максимума.
- **Дисперсия (Variance):** $30\% - 15\% = 15\%$. Основная проблема — это дисперсия. Нужно сосредоточиться на методах снижения дисперсии.

Addressing Bias and Variance / Борьба со смещением и дисперсией

Самая базовая и надежная формула для решения этих проблем выглядит так:

- Если у вас **высокое устранимое смещение** — увеличивайте размер вашей модели (добавляйте слои или нейроны в нейросеть), меняйте архитектуру или модифицируйте входные признаки.
- Если у вас **высокая дисперсия** — добавляйте новые данные в обучающую выборку, используйте регуляризацию (L2, dropout) или раннюю остановку (early stopping).

Bias vs. Variance tradeoff / Компромисс между смещением и дисперсией

В традиционной статистике существовал строгий компромисс: действия, уменьшающие смещение (например, добавление признаков), увеличивали дисперсию, и наоборот. В современную эпоху глубокого обучения и больших данных этот компромисс сильно ослаблен. Если у вас большая нейросеть с правильно настроенной регуляризацией, вы можете безопасно снижать смещение (увеличивая размер сети), не увеличивая при этом дисперсию. А добавляя данные, вы можете снижать дисперсию, не ухудшая смещение.

Techniques for reducing avoidable bias / Методы снижения устранимого смещения

1. **Увеличение размера модели** (больше скрытых слоев/нейронов). Практически всегда снижает смещение. Риск роста дисперсии купируется регуляризацией.
2. **Модификация признаков** на основе анализа ошибок.
3. **Уменьшение или отключение регуляризации** (L2, dropout). Снизит смещение, но поднимет дисперсию.
4. **Изменение архитектуры модели** на более подходящую для данной задачи.

Что НЕ помогает: добавление новых обучающих данных. Сбор данных помогает бороться с дисперсией, но практически не влияет на смещение.

Error analysis on the training set / Анализ ошибок на обучающей выборке

Если ваша модель страдает от высокого смещения, полезно провести анализ ошибок прямо на обучающей выборке. Возьмите ~100 примеров из обучающего набора, на которых алгоритм ошибся, послушайте аудиозаписи или посмотрите картинки вручную. Посчитайте категории: например, 75% ошибок на обучении происходят из-за сильного фонового шума ветра. Это подскажет вам, как изменить архитектуру или признаки модели для лучшей подгонки под сложные обучающие примеры.

Techniques for reducing variance / Методы снижения дисперсии

1. **Добавление большего количества обучающих данных** — самый надежный и простой способ снижения дисперсии, если у вас есть вычислительные ресурсы и доступ к данным.
2. **Добавление регуляризации** (L2, L1, dropout) — снижает дисперсию за счет некоторого контролируемого роста смещения.
3. **Ранняя остановка (Early stopping)** во время градиентного спуска на основе ошибки на dev-выборке.
4. **Отбор признаков (Feature selection)** для уменьшения их количества (актуально при очень маленьких обучающих выборках).

LEARNING CURVES

КРИВЫЕ ОБУЧЕНИЯ

Diagnosing bias and variance: Learning curves / Диагностика смещения и дисперсии с помощью кривых обучения

Кривая обучения — это график, на котором по оси абсцисс откладывается размер обучающей выборки (m), а по оси ординат — уровень ошибки на выборке dev. Чтобы построить такой график, обучите копии вашей модели на подмножествах данных разного объема (например, на 100, 200, 300... 1000 примеров):

По мере роста размера выборки ошибка на dev должна снижаться. Добавив на график горизонтальную пунктирную линию желаемого качества (Desired performance), вы сможете визуальное экстраполировать красную кривую и понять, поможет ли простое удвоение или утроение данных достичь цели, либо кривая уже вышла на плато:

Если красная кривая ошибки dev полностью выровнялась (вышла на плато), это ценнейший сигнал: сбор новых данных бесполезен, нужно менять модель или признаки. В ином случае, если есть явный тренд к улучшению, сбор дополнительных данных имеет смысл:

Plotting training error / Построение кривой ошибки на обучении

В отличие от ошибки на dev-выборке, которая снижается при росте объема данных, ошибка на обучающей выборке обычно **растет** по мере увеличения m . Если у вас всего 2 примера, модель легко зазубрит их до 0% ошибки. Если примеров 10 000, идеально подогнать их все становится намного сложнее, особенно при наличии шума или спорных меток. Добавим синюю кривую ошибки обучения (Training error) на наш график:

Синяя кривая всегда лежит ниже красной кривой dev-ошибки. Совместный анализ обеих кривых позволяет безошибочно диагностировать проблемы высокого смещения или высокой дисперсии, о чем мы подробно поговорим в следующих главах второй половины книги.

Interpreting learning curves: High bias / Интерпретация кривых обучения: Высокое смещение

Предположим, ваша кривая ошибки для dev-выборки вышла на плато, и вы хотите понять, поможет ли добавление данных. Посмотрим на график, объединяющий ошибку на обучении и валидации:

Когда синяя кривая (Training error) находится близко к красной кривой (Dev error), но при этом обе они расположены значительно выше уровня желаемого качества (Desired performance), перед нами классический случай **высокого устранимого смещения (High avoidable bias)**.

В этой ситуации добавление новых данных не поможет, так как синяя кривая при увеличении объема данных может только возрасти или остаться неизменной, отдаляясь от целевой отметки. Необходимо менять саму модель.

Interpreting learning curves: Other cases / Другие случаи интерпретации кривых

Рассмотрим случай высокой дисперсии (High variance):

Здесь ошибка на обучении очень низкая, а ошибка на dev-выборке значительно выше. Между ними существует большой разрыв (gap). В данном случае добавление обучающих данных гарантированно поможет сузить этот разрыв и улучшить качество.

Если же мы наблюдаем картину, где ошибка обучения высока, а разрыв до dev-выборки всё еще велик, мы имеем дело со **смещением и дисперсией одновременно**. Нужно работать над обоими компонентами.

Plotting learning curves / Практические аспекты построения кривых обучения

Построение кривых обучения на малых объемах данных может быть сопряжено с высоким уровнем случайного шума. Чтобы сгладить кривую, можно применить технику многократной случайной выборки с возвращением (sampling with replacement) для каждого фиксированного размера обучающего подмножества и строить график по усредненным значениям ошибки.

Если обучение модели ресурсоемко, вместо линейного шага (1000, 2000, 3000...) используйте геометрический или адаптивный шаг (например, 1000, 2000, 4000, 8000...), чтобы уловить общие тренды с минимальными затратами на вычисления.

COMPARING TO HUMAN-LEVEL PERFORMANCE

СРАВНЕНИЕ С УРОВНЕМ ЧЕЛОВЕЧЕСКОГО

ВОСПРИЯТИЯ

Why we compare to human-level performance / Почему мы сравниваем успехи моделей с человеком

Многие современные ИИ-системы автоматизируют задачи, с которыми люди справляются превосходно. Сравнение с человеком дает нам три мощных преимущества:

1. **Простота разметки:** люди могут легко и быстро генерировать высококачественные метки для обучающей выборки.
2. **Интуитивный анализ ошибок:** если человек справляется с задачей лучше модели, аналитик может понять, какие именно паттерны (контекст, интонация, визуальные детали) модель упускает.
3. **Оценка неустранимой ошибки:** человеческий уровень служит отличным прокси-ориентиром для оценки ошибки Байеса.

How to define human-level performance / Как определить уровень человеческого качества

Предположим, в задаче анализа медицинских снимков обычный человек дает 15% ошибки, молодой врач — 10%, опытный специалист — 5%, а консилиум из нескольких экспертов — 2%. Какую цифру взять за ориентир?

В качестве прокси для оптимальной ошибки (ошибки Байеса) следует выбрать наилучший показатель — 2%. Это даст вам максимальное пространство для оптимизации устранимого смещения модели.

Surpassing human-level performance / Превосходство над человеком

Как только средняя точность вашей модели превосходит человеческий уровень, традиционный анализ ошибок на основе человеческой интуиции становится менее эффективным. В задачах вроде предсказания котировок акций, рекомендаций товаров или логистики машины давно превзошли людей, и прогресс там идет медленнее, так как сложнее нащупать теоретический предел качества.

TRAINING AND TESTING ON DIFFERENT DISTRIBUTIONS

ОБУЧЕНИЕ И ТЕСТИРОВАНИЕ НА РАЗЛИЧНЫХ РАСПРЕДЕЛЕНИЯХ

When you should train and test on different distributions / Сценарии несовпадения распределений

У вас есть 10 000 размеченных фотографий от реальных пользователей приложения и 200 000 идеальных картинок из интернета. Ошибкой было бы перемешать их случайным образом. Обучайте модель на смеси интернет-картинок и части пользовательских данных, но выборки dev и test формируйте **исключительно из реальных пользовательских данных**.

How to decide whether to use all your data / Стоит ли использовать все доступные данные

Если у вас есть огромная емкость модели (крупная нейросеть), добавление данных из другого распределения (например, архивных документов или сторонних сайтов) практически никогда не вредит системе, так как у сети достаточно «места в памяти», чтобы выучить специфику обоих источников без ущерба для целевой задачи.

How to decide whether to include inconsistent data / Проблема противоречивых данных

Если вы предсказываете цены на недвижимость в Нью-Йорке, добавление данных по Детройту (где цены существенно ниже при той же площади) напрямую повредит вашей модели, так как нарушается функциональная зависимость $x \rightarrow y$. Такие противоречивые данные включать нельзя, если только вы не введете явный дополнительный признак города.

Weighting data / Взвешивание данных при обучении

Когда объём дополнительных данных из интернета многократно превышает объём целевых данных, модель может сфокусироваться на оптимизации интернет-ошибки. Чтобы сбалансировать это, введите коэффициент ***eta*** в функцию потерь:

Например, установив ***eta* = 1/40**, вы уравниваете веса целевых и дополнительных примеров:

Generalizing from the training set to the dev set / Обобщение с обучения на dev-выборку

Для точной диагностики в условиях разных распределений вводится дополнительный датасет — **Training-dev set**. Он имеет то же распределение, что и обучающая выборка, но не участвует в обучении параметров. Теперь у вас четыре выборки:

- **Training set:** данные для обучения (Интернет + Мобильные).
- **Training-dev set:** валидация обучения (Интернет + Мобильные).
- **Dev set:** целевая валидация (только Мобильные).
- **Test set:** целевой тест (только Мобильные).

Identifying Bias, Variance, and Data Mismatch Errors / Идентификация типов ошибок и несоответствия данных

С помощью Training-dev выборки мы можем четко локализовать проблему:

- Если ошибка на Training = 1%, на Training-dev = 5%, а на Dev = 5% — у нас классическая **высокая дисперсия**.
- Если ошибка на Training = 1%, на Training-dev = 1.5%, а на Dev = 10% — перед нами чистая проблема **несоответствия данных (data mismatch)**.

Addressing data mismatch / Способы преодоления data mismatch

Проведите анализ ошибок на dev-выборке. Если вы обнаружили, что модель ломается на аудиозаписях, сделанных внутри автомобиля, ваша цель — целенаправленно искать или синтезировать обучающие данные, содержащие дорожные шумы и гул мотора.

Вы можете смешать чистую речь с записью шума автомобиля, получив реалистичный синтетический звук. Будьте осторожны: если у вас всего 1 час уникального автомобильного шума, алгоритм может переобучиться под конкретные частоты этого часа, и синтез не даст ожидаемого эффекта обобщения на реальные автомобили.

DEBUGGING INFERENCE ALGORITHMS

ОТЛАДКА АЛГОРИТМОВ ЛОГИЧЕСКОГО

ВЫВОДА

The Optimization Verification test / Тест верификации оптимизации

В сложных системах (например, распознавании речи) модель вычисляет оценку $\text{Score}_A(S)$, а приближенный алгоритм поиска (например, Beam Search) пытается найти предложение S , максимизирующее эту оценку. Если система вместо корректной фразы S^* выдала ошибочную S_{out} , проведите тест:

Сравните их скоры: если $\text{Score}_A(S^) > \text{Score}_A(S_{\text{out}})$, то виноват поисковый алгоритм (Beam Search пропустил максимум). Если наоборот — виновата сама оценочная функция (модель присвоила плохой фразе высокий балл).*

ГЛАВА 45

General form of Optimization Verification test / Общая форма теста верификации

Этот паттерн применим везде, где есть разделение на функцию оценки качества **Score(y)** и алгоритм оптимизации, ищущий наилучший **y** (например, в машинном переводе с китайского на английский).

При обучении вертолета выполнению опасных маневров без двигателя (авторотация) мы сравниваем траекторию человека T_{human} и алгоритма T_{out} . Если функция награды $R(T_{\text{human}}) \leq R(T_{\text{out}})$, значит, наша математическая формулировка награды неверно описывает реальность, и нужно переписывать функцию $R(\cdot)$.

END-TO-END DEEP LEARNING

СКВОЗНОЕ ГЛУБОКОЕ ОБУЧЕНИЕ

The rise of end-to-end learning / Становление сквозного обучения

Вместо классических многоступенчатых цепочек (Парсер текста *lightarrow* Классификатор тональности) современные системы сквозного обучения (end-to-end) напрямую преобразуют исходный текст в итоговый класс позитивного/негативного отзыва с помощью единой нейросети.

More end-to-end learning examples / Дополнительные примеры сквозных систем

В распознавании речи сквозная архитектура заменяет цепочку из извлечения MFCC-признаков, распознавания фонем и лингвистического классификатора прямой картой (Аудиозапись *ightarrow* Текст транскрипта):

Для беспилотных автомобилей классический подход включает детекторы машин, пешеходов и модуль планирования траектории. Сквозной метод пытается отобразить картинку с камеры напрямую в угол поворота руля:

Pros and cons of end-to-end learning / Плюсы и минусы сквозного подхода

Главный плюс сквозного обучения — нейросеть не ограничена рамками человеческих понятий (вроде фонем или жестких геометрических правил) и может найти скрытые оптимальные взаимосвязи. Главный минус — для обучения сквозных карт требуется колоссальное, часто недостижимое количество размеченных пар «вход-выход».

Choosing pipeline components: Data availability / Выбор компонентов пайплайна: Доступность данных

Разбивать систему на промежуточные блоки выгодно тогда, когда для каждого блока в отдельности на рынке есть огромные массивы данных (например, миллионы размеченных силуэтов пешеходов и машин для компьютерного зрения), в то время как готовых датасетов сквозного вождения крайне мало.

Choosing pipeline components: Task simplicity / Выбор компонентов пайплайна: Простота подзадач

Всегда стремитесь декомпозировать сложную комплексную задачу на серию более простых, легко поддающихся обучению изолированных функций. Например, детектировать сиамскую кошку проще в два этапа: (1) Найти кошку на кадре, (2) Передать вырезанный фрагмент классификатору пород.

Directly learning rich outputs / Прямое обучение сложным комплексным ВЫХОДАМ

Современные глубокие архитектуры способны выдавать на выходе не просто одно число, а богатые структурированные объекты: целые текстовые описания к картинкам, аудиопотоки (TTS) или готовые переводы предложений.

Задача	Вход (X)	Выход (Y)
Генерация описаний	Изображение	Текст описания
Машинный перевод	Текст на английском	Текст на французском
Распознавание речи	Аудиозапись	Текстовый транскрипт

ERROR ANALYSIS BY PARTS

ПОКОМПОНЕНТНЫЙ АНАЛИЗ ОШИБОК

ПАЙПЛАЙНА

В многокомпонентном пайплайне детектора сиамских кошек мы можем точно установить, какой модуль подвел систему на ошибочном примере: детектор кошек (вырезал пустые камни) или классификатор пород.

Attributing error to one part / Точное приписывание вины конкретному модулю

Если на вход классификатора пород пришел некачественно обрезанный снимок, проведите эксперимент: подставьте вместо выхода детектора идеальную рамку, размеченную вручную человеком. Если классификатор теперь выдал верный ответ — вина полностью лежит на детекторе.

General case of error attribution / Общий случай покомпонентного анализа ошибок

Для последовательного пайплайна $A \rightarrow B \rightarrow C$ мы поочередно заменяем выходы модулей на эталонные человеческие метки. Первый модуль, при замене выхода которого итоговая система начинает работать идеально, объявляется основным источником ошибки.

Измерьте, насколько далеко качество работы каждого отдельного изолированного модуля (например, детектора пешеходов) находится от аналогичного человеческого показателя. Инвестируйте время инженеров в тот модуль, у которого отставание от человека максимально.

Spotting a flawed ML pipeline / Как обнаружить фундаментальные дефекты архитектуры пайплайна

Если каждый отдельный компонент вашего беспилотника (детектор машин, пешеходов, разметки) работает на уровне человека, но машина при этом водит крайне плохо — значит, сама структура вашего пайплайна дефектна. Входных признаков недостаточно, и планировщику траектории, например, критически не хватает данных о дорожных знаках или сигналах светофора. Пайплайн требует редизайна.

Building a superhero team / Построение команды супергероев

Поздравляю вас с завершением этой книги! Самое лучшее после того, как вы сами научились безошибочно определять стратегию развития ИИ-проектов — поделиться этими краткими главами со своими коллегами, создав настоящую сплоченную команду супергероев машинного обучения.